

docs.google.com – 全画面表示を終了するには、Esc を押します

TL1

01_12.ローダーと配置

日本工学院専門学校
デザインカレッジ

再帰関数の実装例（擬似コード）

日本工学院専門学校
デザインカレッジ

```
構造体 Object:
  文字列 name
  リスト<Object> children

関数 ConvertJsonToObjects(jsonNode) -> Object:
  // JSONノードの情報を元にObjectを作成
  新しいノード object = 新しい Object
  object.name = jsonNode["name"]
  object.children = 空のリスト

  // 子ノードが存在する場合、再帰的に処理
  子リスト = jsonNode["children"]
  もし 子リスト が存在するなら:
    各 子json in 子リスト に対して:
      子object = ConvertJsonToObjects(子json)
      object.children に 子object を追加

  return object
```

ツリー構造の走査部分についての
擬似コードによる実装例。

要はjsonのツリー構造を走査しながら
jsonのツリー構造から
自作クラス（構造体）のツリー構造に
ノード（オブジェクト）の情報を
移し取っていくのがポイント。

※最悪、親子構造はゲーム側に再現できな
くても合格とする

Appendix.

注意点

日本工学院専門学校
デザインカレッジ

実際のゲーム制作において

「エディタ上では球やボックスをダミーとして配置しておいて

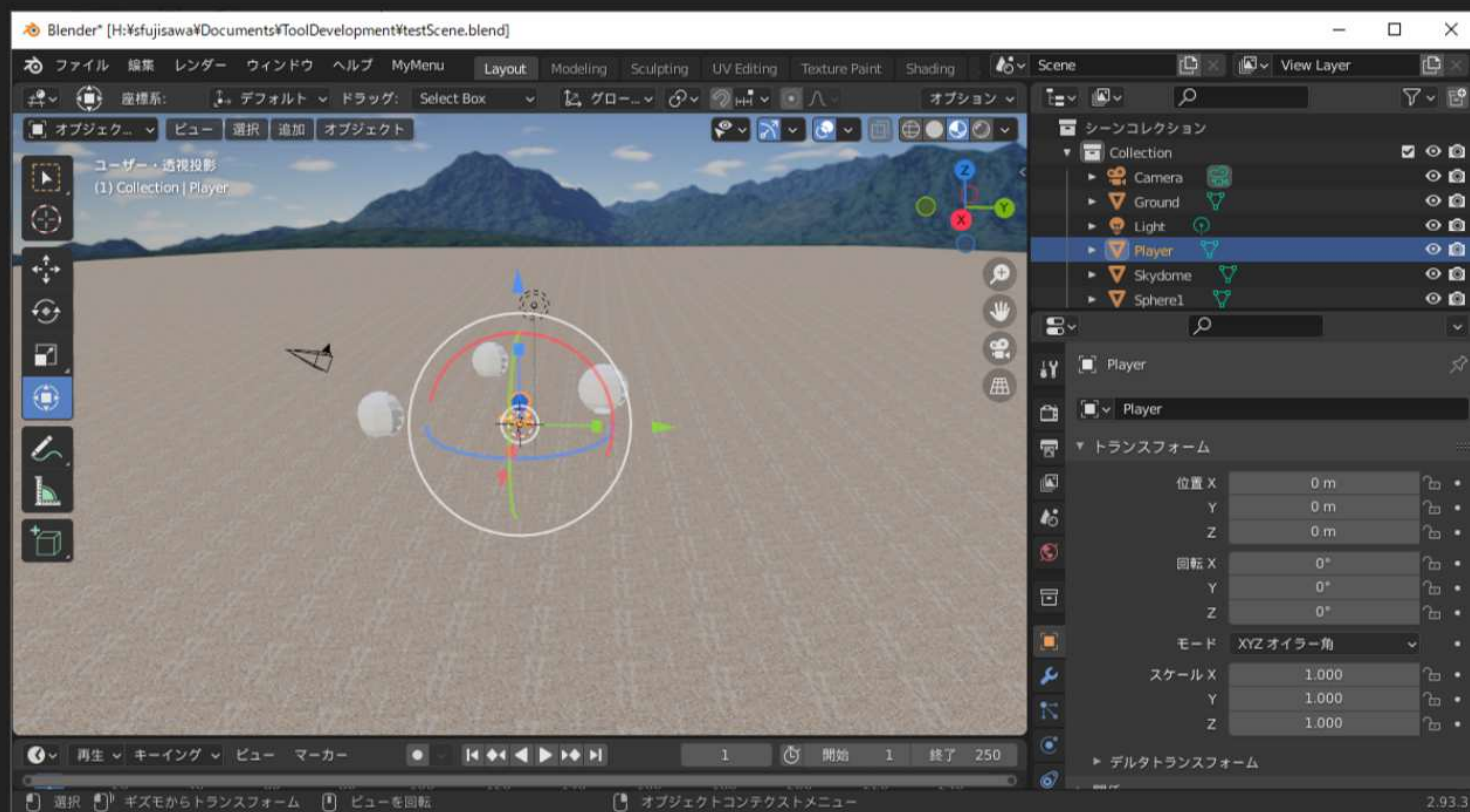
ゲーム画面ではちゃんとしたモデルが出ているからいいよね」では不便すぎて使い物にならない。

ライティングやテクスチャは別としても、配置物の形状はエディタ（Blender）上とゲームで同じ見た目になっていて、はじめて完成といえる点に注意しよう。

完成確認はBlenderで直接できるボックス、球などの各種基本形状でなく、
適当に整形した .obj や .fbx , .glTF などをインポートして配置し、確認すること。

完成

日本工学院専門学校
デザインカレッジ

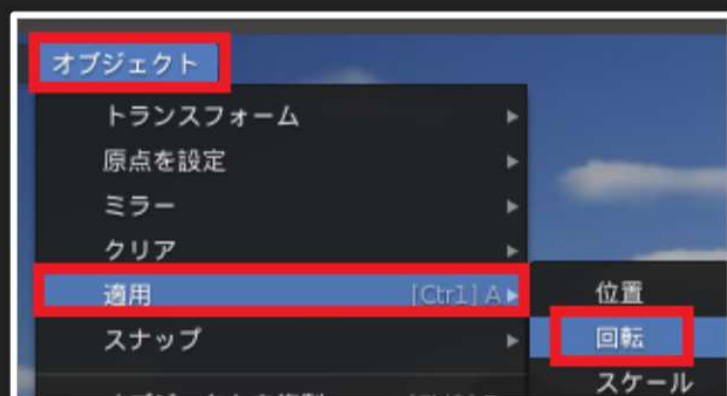


ゲーム内の見た目に
近い状態でBlenderの
配置をエディットできる
ようになった。

あとはレベルエディタ、
ローダーともに機能をさら
に拡張して、より使いやす
く便利にしていこう。

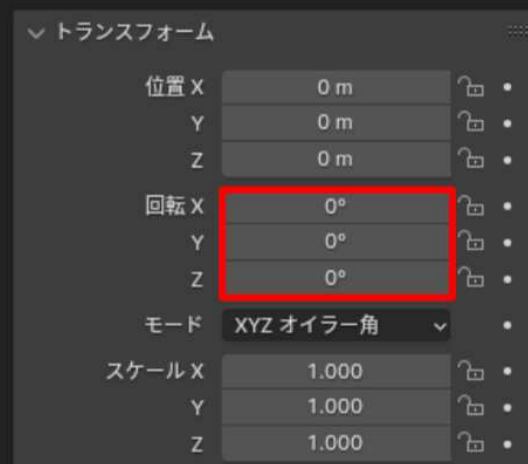
Blenderでの配置の注意点

日本工学院専門学校
デザインカレッジ



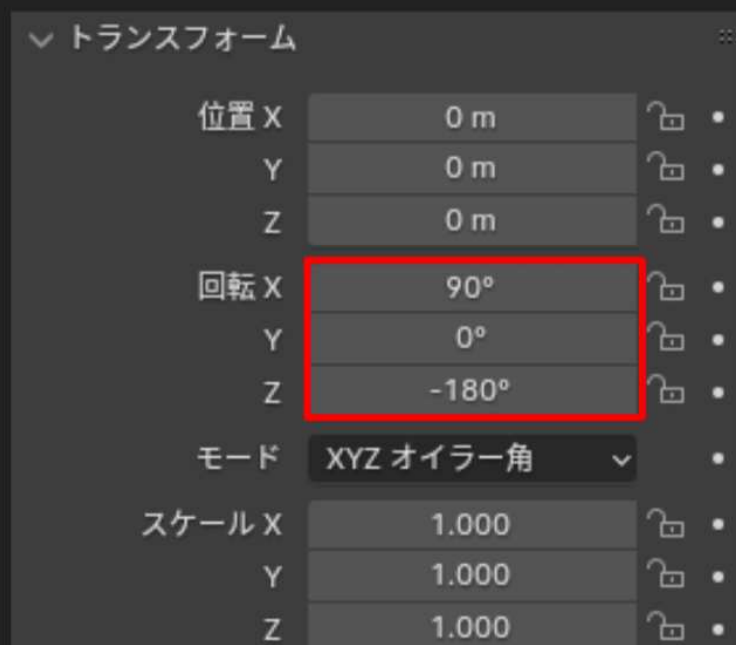
OBJをBlenderにインポートした時点で
3Dビューから「オブジェクト」→「適用」→「回転」を
実行しておく。

こうすることで、見た目の角度はそのままに
回転角が(0,0,0)にリセットされる。



Blenderでの配置の注意点

日本工学院専門学校
デザインカレッジ



Blender でOBJをインポートするとき、
軸方向はエクスポート時の設定に合わせる。
(前方:Z, 上:Y)

Blender でOBJをインポートすると、
左図のように、回転Xに90、回転Zに-180
という値が入った状態になる。
(見た目の角度はこれで合っている)

このまま配置してレベルデータを出力すると
無回転状態で見た目が一致していたはずなのに、
X90度回転、Z-180度回転がゲーム上で余分に適用されて
おかしくなる。

(何を言っているか分からないと思うので、実際にゲームで読み
込んで試してみよう)

シーンに配置

日本工学院専門学校
デザインカレッジ

```
// レベルデータからオブジェクトを生成、配置
for (auto& objectData : levelData->objects) {
    // ファイル名から登録済みモデルを検索
    Model* model = nullptr;
    decltype(models)::iterator it = models.find(objectData.fileName);
    if (it != models.end()) { model = it->second; }
    // モデルを指定して3Dオブジェクトを生成
    Object3d* newObject = Object3d::Create(model);
    // 座標
    newObject->SetPosition(objectData.translation);
    // 回転角
    newObject->SetRotation(objectData.rotation);
    // 座標
    newObject->SetScale(objectData.scaling);
    // 配列に登録
    objects.push_back(newObject);
}
```

LevelData 型にまとめた配置データをもとに、実際にオブジェクトを生成し、シーンに配置していく。

なお、サンプルでは決め打ちでモデルを読み込んでモデルリスト(std::map)に登録しているが、モデルリストもローダーがLevelData型内にまとめておき、シーン側がfor文で読み込むようにするとよい。

オブジェクトの配置

日本工学院専門学校
デザインカレッジ

- レベルデータ通りにオブジェクトを配置する
をやっていく。

ここからはシーン制御クラスの担当。

オブジェクトの配置

ツリー構造の走査

日本工学院専門学校
デザインカレッジ

```
// "objects"の全オブジェクトを走査
for (nlohmann::json& object : deserialized["objects"]) {
    assert(object.contains("type"));

    // 種別を取得
    std::string type = object["type"].get<std::string>();

    種類ごとの処理

    // TODO: オブジェクト走査を再帰関数にまとめ、再帰呼出で枝を走査する
    if (object.contains("children")) {
    }
}
```

ツリー構造の走査は再帰呼び出しが基本。

9ページ の全オブジェクト走査の部分から関数に抜き出して再帰呼び出しを行う。

サンプルではやっていないが、ぜひやるべき。

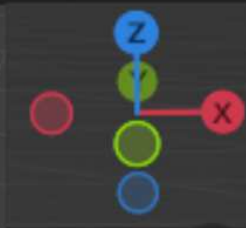
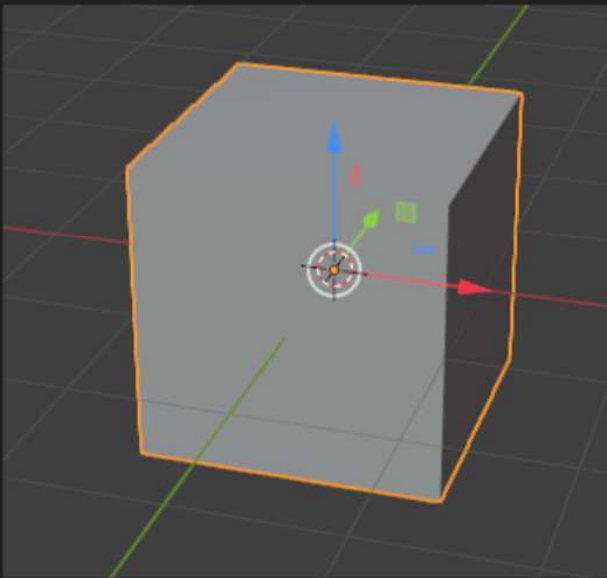
※reserveしないとvectorの引っ越しが発生する危険性あり。

ローダーの処理はここまでで終わり。

LevelData 型にまとめた配置データをシーン制御クラスに渡す。

トランスフォームのパラメータ

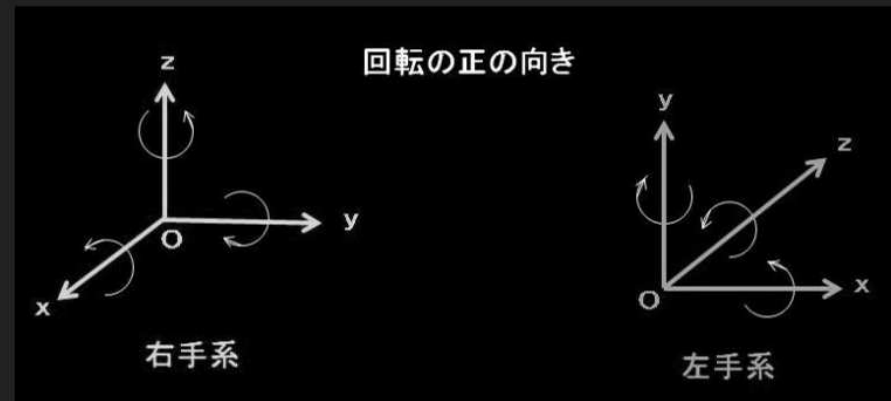
日本工学院専門学校
デザインカレッジ



Blenderでは

- Xが右 (ゲーム:X に Blender: X を代入)
- Yが奥 (ゲーム:Z に Blender: Y を代入)
- Zが上 (ゲーム:Y に Blender: Z を代入)

となる。



トランスフォームのパラメータ

日本工学院専門学校
デザインカレッジ

```
// トランスフォームのパラメータ読み込み
nlohmann::json& transform = object["transform"];
// 平行移動
objectData.translation.x = (float)transform["translation"][0];
objectData.translation.y = (float)transform["translation"][2];
objectData.translation.z = (float)transform["translation"][1];
// 回転角
objectData.rotation.x = -(float)transform["rotation"][0];
objectData.rotation.y = -(float)transform["rotation"][2];
objectData.rotation.z = -(float)transform["rotation"][1];
// スケーリング
objectData.scaling.x = (float)transform["scaling"][0];
objectData.scaling.y = (float)transform["scaling"][2];
objectData.scaling.z = (float)transform["scaling"][1];
```

トランスフォームの各パラメータ
「平行移動」「回転角」
「スケーリング」を読み込む。

この時、Blenderと自分のゲームの
座標系の軸方向の違いを吸収して
おく。

軸方向変換は、ここでやる方法と
エクスポーターでやる方法の
二通りある。
(エクスポーターでやることが多い)

メッシュの読み込み

日本工学院専門学校
デザインカレッジ

```
// MESH
if (type.compare("MESH") == 0) {
    // 要素追加
    levelData->objects.emplace_back(LevelData::ObjectData{});
    // 今追加した要素の参照を得る
    LevelData::ObjectData& objectData = levelData->objects.back();

    if (object.contains("file_name")) {
        // ファイル名
        objectData.fileName = object["file_name"];
    }

    トランスフォームのパラメータ読み込み

    // TODO: コライダーのパラメータ読み込み
}
```

type が MESH である場合の処理。

「読み込んだレベルデータ」の
オブジェクトリストに追加する。
(本当はMESHを含めた全オブジェクト
を追加すべき)

コライダーのパラメータは
サンプルでは使っていない。
自分で読み込みと使用を実装しよう。

オブジェクトの走査

日本工学院専門学校
デザインカレッジ

```
// レベルデータ格納用インスタンスを生成
LevelData* levelData = new LevelData();

// "objects"の全オブジェクトを走査
for (nlohmann::json& object : deserialized["objects"]) {
    assert(object.contains("type"));

    // 種別を取得
    std::string type = object["type"].get<std::string>();

    種類ごとの処理

    再帰処理
}
```

この時点では、木構造は再現されていても「JSONから読み込んだ汎用的なデータ」に過ぎないので、自分のゲームに使えるように整理していく。

LevelData 型の変数は整理して格納するための入れ物として用意した。

Blenderから出力した
オブジェクトリストを走査する。

各オブジェクトには必ず "type" データを入れているので、"type" が検出できなければ不正として実行を停止する。

ファイルチェック

日本工学院専門学校
デザインカレッジ

```
// JSON文字列から解凍したデータ
nlohmann::json deserialized;

// 解凍
file >> deserialized;

// 正しいレベルデータファイルかチェック
assert(deserialized.is_object());
assert(deserialized.contains("name"));
assert(deserialized["name"].is_string());

// "name"を文字列として取得
std::string name =
deserialized["name"].get<std::string>();
// 正しいレベルデータファイルかチェック
assert(name.compare("scene") == 0);
```

ifstream でファイルを開いたら、>> 演算子で
nlohmann::json 型の変数にデータを全部流し込む。

この時点で内部に木構造が再現されている。

[01_11]の表でしめした通り、
json のノードは8種類あるが
その内の「object」は連想配列で、std::map と同じよう
な使い方ができるようになっている。

最初に、読み込んだファイルが本当にレベルデータファイ
ルかチェックする。

出力ファイルの先頭に何を配置するかは自分で決めている
ので、その通りのデータが先頭になれば
不正なレベルデータファイルとして検出する。

JSONファイルを読み込んでみる

日本工学院専門学校
デザインカレッジ

```
// 連結してフルパスを得る
const std::string fullpath = kDefaultBaseDirectory + fileName + kExtension;

// ファイルストリーム
std::ifstream file;

// ファイルを開く
file.open(fullpath);
// ファイルオープン失敗をチェック
if (file.fail()) {
    assert(0);
}
```

まずは `ifstream` でJSONファイルを開く。

JSONライブラリ

日本工学院専門学校
デザインカレッジ

[01_11]で、出力するレベルデータファイルをJSONテキストにした。

JSONは大抵のプログラミング言語で誰かがライブラリを作っている。

今回はC++言語用の [nlohmann json](#) を使用する。
(ローマンジェイソン)

「[Tags](#)」から最新版をダウンロードして組み込む。
.cppと.hppがセットになっているものがあるが、
ヘッダーファイル(.hpp) 1つのみ版は組み込みがお手軽でオススメ。
各バージョンの「Downloads」でバージョンの詳細ページに飛び、
詳細ページ下部の「Assets」から「json.hpp」を選択してダウンロードする。

JSONの書式はそこまで複雑なわけではないので、
勉強がてら外部ライブラリを使わず自分でローダーを書くのもよい。

レベルデータローダー

日本工学院専門学校
デザインカレッジ

- ローダーを作ってレベルデータをファイルから読み込む
をやっていく。

レベルデータローダー

ローダーと配置

日本工学院専門学校
デザインカレッジ

Blenderをレベルエディタとして使えるように機能拡張したが、ゲーム側で

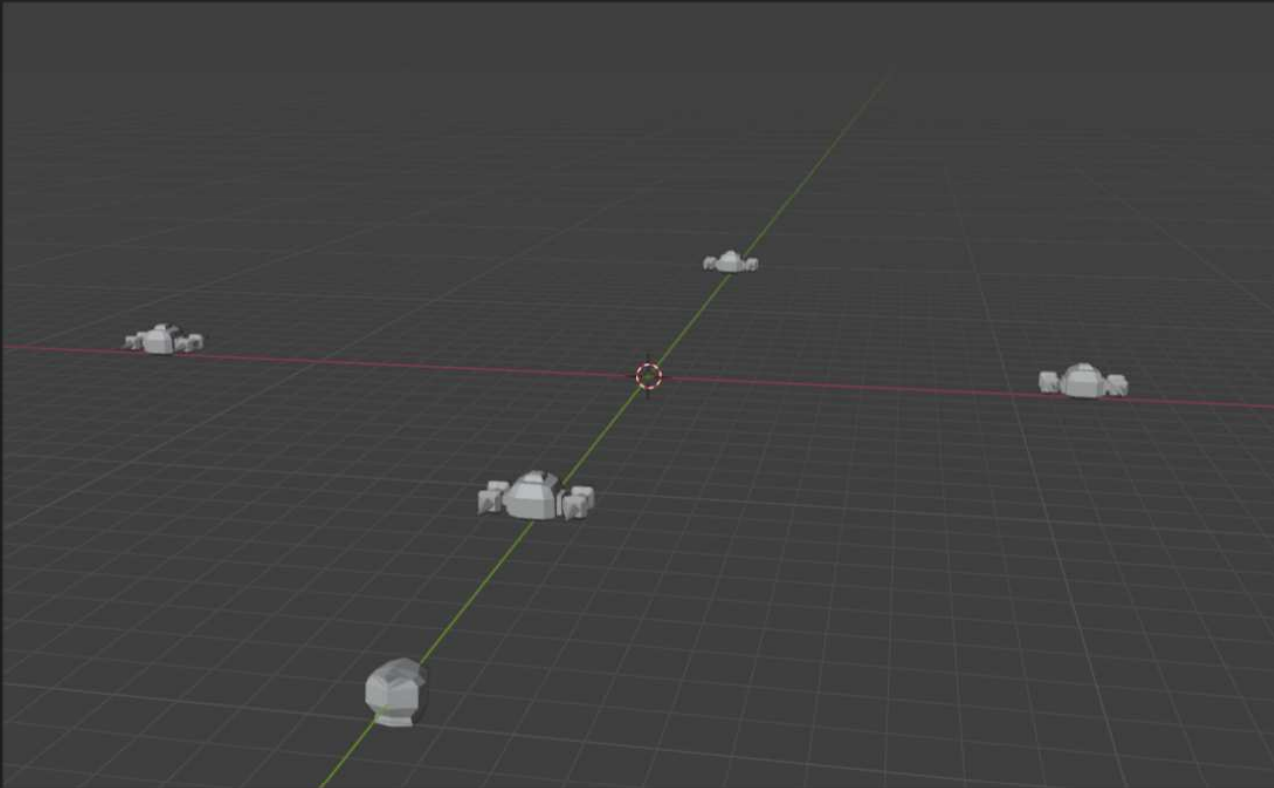
- ローダーを作ってレベルデータをファイルから読み込む
- レベルデータ通りにオブジェクトを配置する

をやってはじめてゲームに投入できる。

最低限のところまでを示すので、あとは自分で改造してほしい

今回の目標

日本工学院専門学校
デザインカレッジ



フィールド上での
自キャラや敵の出現位置を
Blender上で配置したり、
1つ1つの敵の種類を
Blender上で設定できる。

TL1

01_12.ローダーと配置

日本工学院専門学校
デザインカレッジ